HEAPS & PRIORITY QUEUES

"Not everyone waits their turn — the most important one always goes first."

▪ TODAY'S CLASS ▪

Last class we finished BSTs. Today we meet a new structure that does NOT care about full ordering — it only cares about ONE thing at a time: who comes next. Lighter, simpler, and used everywhere in production.

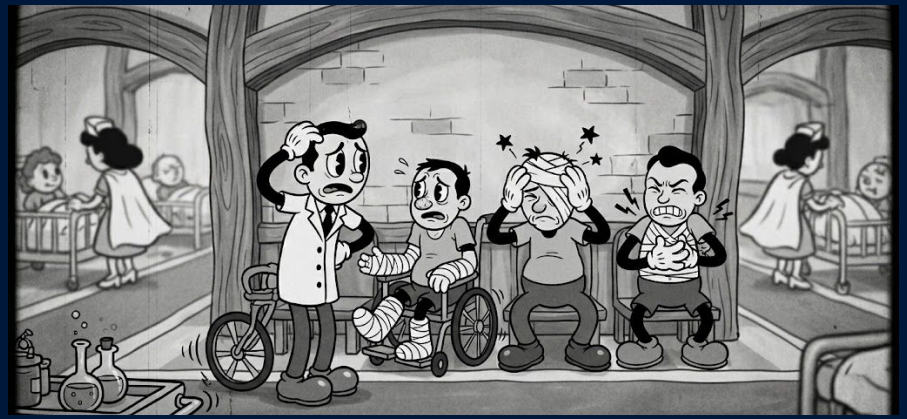
Block	What we cover
The heap	Definition, min vs max
Array trick	Tree without pointers
Push	Add + bubble up
Pop	Extract + bubble down
`heapq`	Python's built-in min-heap

"No full ordering. Just one rule: parent before children. That alone covers half the systems you use."

PRIORITY OVER ORDER

You walk into a hospital ER (急诊室). Three people are waiting: a man with a cut finger, a woman with a broken arm, and a patient having a heart attack. Who goes first?

Not the one who arrived first — the one with the highest priority. That is a **priority queue**: returns the most important element first, no matter when it was added. We need a fast structure for it. Today: the **heap**.



[01] Task scheduling
Your phone runs the urgent system update before the Bilibili upload.

[02] Top-K recommendations
Douyin keeps the K best videos for you, right now, not the K oldest.

[03] Dijkstra's algorithm
Baidu Maps and Gaode Maps pop the closest unvisited node every step.

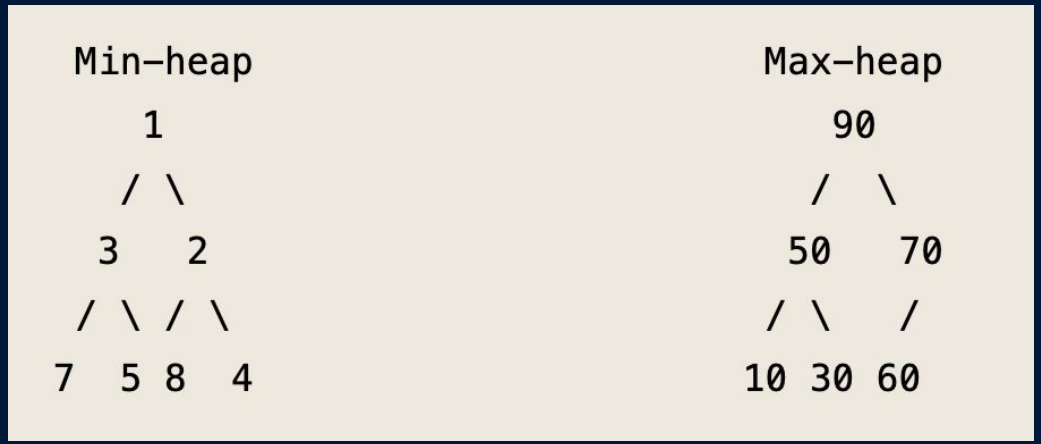
[04] Hospital triage, airport boarding
First / Business / Economy — priority decides the order.

"Queues serve the oldest. Priority queues serve the most important. That difference shows up everywhere."

THE HEAP PROPERTY

A binary heap is a **complete binary tree** with one rule. In a min-heap, every parent is smaller or equal to its children — so the root is the minimum. In a max-heap it's the opposite.

Complete means filled level by level, left to right, no gaps.



This is NOT a BST.

There is no left-right ordering. Only parent ≤ children (min-heap) or parent ≥ children (max-heap).

A heap is much weaker than a BST, and that's the point.

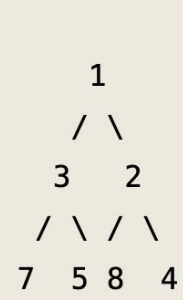
"One rule, between parent and children. No order between siblings. Cheap to maintain."

THE ARRAY TRICK: INDEX ARITHMETIC

Because a heap is a **complete** tree, we can store it in a plain array.
Parent and child positions are simple index formulas.

No left or right pointers, no node, no tree class, just a list and arithmetic.

Min-heap as a tree:



Same heap as an array:

index:	0	1	2	3	4	5	6
value:	1	3	2	7	5	8	4

[01] Parent of `i`

`(i - 1) // 2`

[02] Left child of `i`

`2 * i + 1`

[03] Right child of `i`

`2 * i + 2`

[04] Why it works

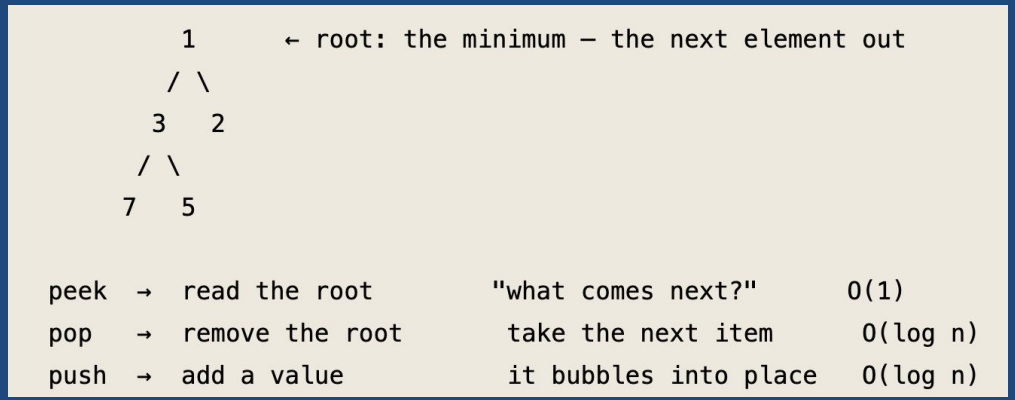
No pointers, No node,O(1) access, cache-friendly contiguous memory.

"No pointers. The shape of the tree is implied by the position in the array."

THE ROOT IS WHO'S NEXT

We now have a complete tree, the heap property, and array indexing. So what does all that machinery buy us? One guarantee — and it is the one that matters: **the root is always the highest-priority element** (the minimum in a min-heap).

A heap never fully sorts its data; it only keeps the most important element on top, cheaply, as values come and go. That single guarantee is exactly what a priority queue is — and it is what the ER doctor needed: the next patient, on demand.



[01] No full ordering needed

A priority queue answers only one question: which element comes next? Fully sorting would be wasted work.

[02] The root is that answer

The heap property guarantees the most important element sits at the root — read in `O(1)`.

[03] It stays true as data changes

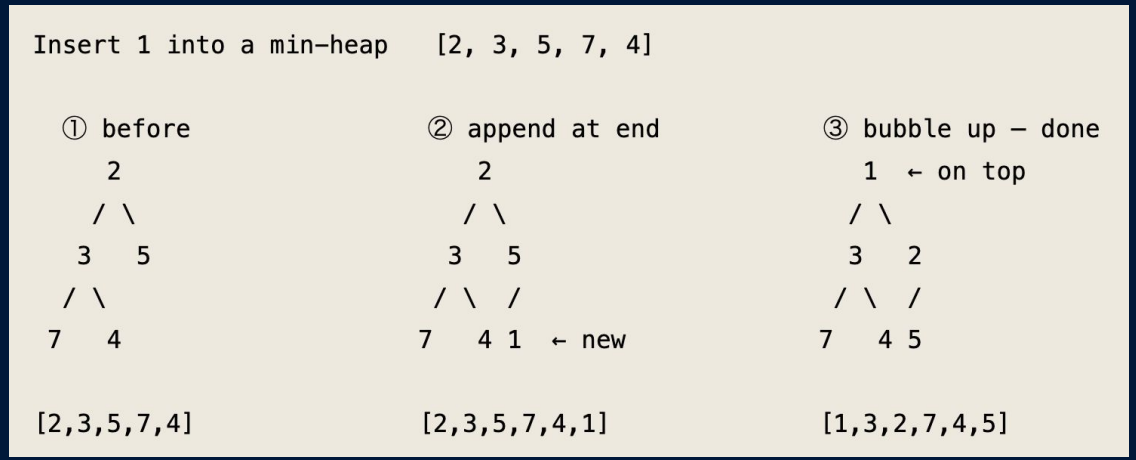
`push` and `pop` keep the "next one out" always correct at `O(log n)` per update.

Heap vs Sorted List: A sorted list pays to order every element; a heap promises only the top is the next one out — at O(log n) per change.

push and bubble up

A new patient arrives — we `push`. Drop the new value into the next free slot at the end of the array, then let it **bubble up**: while it is smaller than its parent, swap.

It climbs until its parent is smaller, or it reaches the root.



[01] Append at the end

Drop the new value 1 into the next free slot — index 5. The tree stays complete.

[02] Compare with the parent

Parent of index 5 = `(5-1)//2` = 2 → value 5. Since 1 < 5, swap.

[03] Keep bubbling up

Parent of index 2 = `(2-1)//2` = 0 → value 2. Since 1 < 2, swap. Now at the root → stop.

"Drop the new value at the end. Let it climb to its level. That's all push does."

pop and bubble down

The doctor calls the next patient — we `pop` the root, the minimum. Move the last element into the empty root, then let it **bubble down**: while it is larger than its smaller child, swap.

It sinks until both children are larger, or it has none.

```
Pop from a min-heap   [1, 3, 2, 7, 4, 5]   →  returns 1

  ① before                ② last → root           ③ bubble down — done
     1  ← returns             5  ← moved up           2  ← new min
    / \                     / \                     / \
   3   2                   3   2                   3   5
  / \  /                  / \                     / \
 7   4 5                 7   4                   7   4

[1,3,2,7,4,5]           [5,3,2,7,4]             [2,3,5,7,4]
```

[01] Save the root

The minimum always sits at the root. Save 1 — that is the value `pop` returns.

[02] Move the last to the root

Take the last element (5) and place it at the root so the tree stays complete.

[03] Bubble down

Compare 5 with its smaller child (2). Since 5 > 2, swap. 5 now has no children → stop.

Complexity: O(log n). Reading the root is O(1); the cost is repairing the heap on the way down. The smaller child always pulls the value down.

"Take the root, plug the gap with the last element, let it sink. The smallest child always pulls it down."

▪ THE COSTS ▪

A heap is fast at exactly what a priority queue needs: peek at the most important element, add a new one, and remove the most important one. Full sorting is not part of the deal.

Operation	Complexity
`peek` (look at min)	**O(1)**
`push`	**O(log n)**
`pop`	**O(log n)**
Build heap from list of n *(trick)*	**O(n)**
Find an arbitrary value	O(n)

A heap is NOT good at finding arbitrary values — only the root is fast. If you also need "search for value X", reach for a BST instead.

"Peek is free. Push and pop are log. Search is bad. Match the tool to the job."

HEAPQ — BUILT-IN

Python gives you a heap for free. The `heapq` module operates **IN-PLACE** on a regular list. It is a min-heap by default. Three core calls, two top-K helpers.

```
import heapq

nums = [5, 3, 8, 1, 9, 2]
heapq.heapify(nums) # list to heap — O(n)
print(nums) # [1, 3, 2, 5, 9, 8] (heap order)

heapq.heappush(nums, 0) # insert 0 — O(log n)
smallest = heapq.heappop(nums) # remove min — O(log n)

# Top-K helpers
top_3 = heapq.nlargest(3, [4, 1, 7, 3, 8, 2, 9])
bottom_3 = heapq.nsmallest(3, [4, 1, 7, 3, 8, 2, 9])
```

Efficiency Boost:

`nlargest(k, lst)` runs in **O(n log k)**.

Much cheaper than sorting the whole list when *k* is small. The workhorse of top-K problems.

"Three calls cover almost everything: heapify, heappush, heappop. nlargest and nsmallest for top-K."

▪ DIFFERENTS TREE STRUCTURES ▪

Heaps and BSTs are both tree-based, but they serve very different jobs. Reach for the one that matches the operations you do most.

	Heap	BST
Ordering	Parent vs children only	Full left/right ordering
Find min	**O(1)**	O(h)
Find arbitrary	O(n)	O(h)
Storage	Array (compact)	Nodes with pointers
Use case	Priority queue	Sorted lookup, queries

BST = "I will search for arbitrary values often."
Heap = "I only ever ask for the most important one." Different tools, different jobs.

"BST orders everything. Heap orders just enough. Match the structure to the question you keep asking."